



De La Salle University – Manila

College of Computer Studies – Computer Technology

CSINTSY - DECLARATION OF AI USAGE

I. BORBE, KEN MARTHIN H. - [AI TOOL(S) USED: CHATGPT AND CLAUDE]

1. Describe briefly how you used the AI tool(s) in the completion of this assessment task. Examples include: brainstorming, idea generation.

1.1 AI used in Code Creation:

- I used AI to understand my groupmate's parts
- I used AI to debug and give ideas for test bots for hidden cats in the future

1.2 AI used in Script Making:

- I used AI to help me with better terminology when explaining my code to groupmates

1.3 AI used in Overall Algorithm:

- I used AI to better understand my groupmates' code so I can integrate their code with mine. I also used AI to give me ideas on hidden bots or test bots to check if it can handle it well

2. In 2–3 sentences, reflect on how the use of AI contributed to your learning. If no AI was used, reflect on why you chose not to.

Using AI helped me understand the parts of the project that I did not implement. It made it easier to integrate my code with my groupmates' work. It also helped me think of additional test cases and improve how I explained technical concepts, but I still had to verify the suggestions and apply my own understandings to verify my own work.

3. Chat Logs:

What are some hidden bot ideas I can use to test my robustness (any weird cat) just give me the idea

7:34 AM

Brainstormed unconventional cat behaviors for testing robustness

Here are some weird cat ideas you could try, just as concepts:

1. **Copycat** - mimics CatBot's last move. If you went up, it also moves up. Tests whether the bot can catch something that "follows" it instead of running.
2. **Coin Flip** - on every move, 50% chance it moves toward CatBot (dumb/suicidal), 50% chance it flees. Totally unpredictable, no fixed pattern.
3. **Sleeper** - stays completely still for the first several moves (like it's napping), then suddenly starts fleeing once CatBot gets close. Tests if the bot capitalizes on early opportunities instead of hesitating.
4. **Teleporter** - small chance each turn to jump to a random tile instead of moving normally. Tests how well the bot recovers when its target "vanishes"
5. **Mirror Cat** - always moves to the tile that's symmetrically opposite CatBot on the grid (mirrored across the center). Very geometric, non-random pattern.
6. **Corner Camper** - beelines for the nearest corner and just sits there, only moving if CatBot gets adjacent. Tests whether the bot can finish off a cat in a tight cornered spot instead of just approaching-and-bumping.
7. **Herd Instinct** - tries to stay near walls/edges but also keeps some minimum distance from CatBot, so it "slides" along edges while doc

whats some better terminology I can use instead of sayin the code moves 10 steps at random but does not count in training the bot so results are random

Excavated alternative terminology for random initialization steps

Good instinct to want cleaner wording. A few options, from most to least common:

1. **"Random start-state injection"** — probably the clearest one. It says exactly what's happening: you're injecting randomness into where the episode starts, not training on those steps.
2. **"State randomization steps"** — simple and plain, no jargon.
3. **"Burn-in"** (what you're already using) is actually legit, standard terminology — it's borrowed from statistics/MCMC, where "burn-in" means throwing away the first batch of samples because they're not representative yet. It's a fine word to keep using, it's just that people unfamiliar with it might not immediately know it means "steps that don't count!"
4. **"Untrained random walk"** — good if you want to emphasize the "does not count toward training" part specifically.
5. **"Pre-episode scrambling"** — casual, easy to say out loud when explaining to someone.

If you want one phrase that captures *both* halves of what you told me (moves randomly + doesn't count for training), I'd combine two of 'em:

— "a random start-state injection — CatBot takes some random moves before the episode"

II. **MOJICA, MAURIENNE** - [AI TOOL(S) USED: CLAUDE]

4. Describe briefly how you used the AI tool(s) in the completion of this assessment task. Examples include: brainstorming, idea generation.

1.1 AI used in Code Creation:

- I used AI to suggest me the best and feasible way to implement the reward structure and state handling logic in training.py so that it would be compatible with the other files that will be used by my groupmates.
- I also used it to double check if my work was correct and for debugging purposes.

1.2 AI used in Script Making:

- I used AI to improve my points for the video recordings in the demo and ownership video to keep it concise and reduce redundancy.

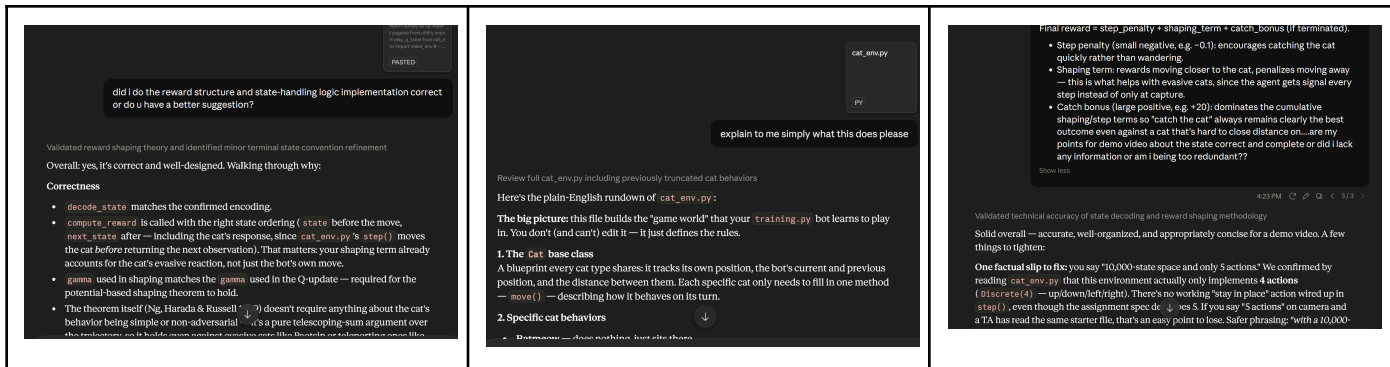
1.3 AI used in Overall Algorithm:

- I used AI to help me explain some of my groupmates parts, and other implementations that I find confusing in order to accomplish the reward structure and state handling logic correctly.

5. In 2–3 sentences, reflect on how the use of AI contributed to your learning. If no AI was used, reflect on why you chose not to.

- Understanding the given files and how our parts as a group connect altogether was challenging, but AI helped me quickly understand their parts and the given files, and suggested me the best way on how my approach should be in implementing the state that would not cause conflict or difficulty to my groupmates. Debugging with AI also taught me on how to fix certain cases where I am not familiar with which greatly improved my understanding of the project and how it works altogether.

6. Chat Logs:



III. PARADO, SKY HANNAH G. - [AI TOOL(S) USED: CHATGPT AND CLAUDE]

7. Describe briefly how you used the AI tool(s) in the completion of this assessment task. Examples include: brainstorming, idea generation.

1.1 AI used in Code Creation:

- I asked AI to explain my part and help me understand what resources to search to gain further understanding.
- When I got stuck and could not debug, I asked AI to enumerate the error

1.2 AI used in Script Making:

- I asked AI to assist me in understanding my groupmates parts.
- Asked AI to check my script and give its thoughts on improvements and suggestions [especially with terminology etc]

1.3 AI used in Overall Algorithm:

- Asked for sources to read and implementation suggestions and comments. The entire program and paper were cross checked by AI, in case there were gaps in knowledge, code, etc. AI ensured we tackled all the specs and split the work evenly

8. In 2–3 sentences, reflect on how the use of AI contributed to your learning. If no AI was used, reflect on why you chose not to.

AI helped me divide the tasks equally among my groupmates, provided internal deadlines in order to ensure fairness and order, allowing for easy asynchronous work until the ownership session which proves our understanding and demo which showcases our final product. AI helps with efficiency especially in dead ends. It allowed us to access more resources in a short amount of time and explained to us the different terminology and things we needed to know in a simple way, which allowed me to be able to work faster and more efficiently.

9. Chat Logs:

does everything work and match specs?

- Short answer: **mostly yes, but not fully verified — and I just found and fixed a crash.** Let me be precise about what I've actually confirmed versus assumed.

Verified working

Check	Status
<code>utility.py</code> , <code>play.py</code> , <code>bot.py</code> , <code>requirements.txt</code> unmodified	✓ zero diff vs original upload
<code>cat_env.py</code> changes confined to <code>TrainerCat</code>	✓ all 45 diff lines inside the class
<code>training.py</code> edits inside designated regions only	✓ audited against all 6 markers
5000-episode cap untouched	✓ training.py:145
Rewards computed in <code>training.py</code> , not <code>cat_env.py</code>	✓
Training under 20s	✓ measured 1.7–9.2s
Runs against a pristine <code>cat_env.py</code>	✓ batmeow 100%, paotsin 100%, squiddyboi 99%
Catches the 5 visible cats	✓ 96.5–100%

Task division system

How many members are in your group?
4

Architected project deliverables and algorithmic validation framework

Here's a task breakdown for CatBot (MCO3), following the same style as your reference doc.
Release Oct 24 → Due Nov 28, so I spread internal deadlines across that ~5-week window.

Role Description	Internal Deadline	Member
[Setup] Environment Familiarization (shared) — Everyone runs <code>play.py</code> on all 5 known cats (batmeow, mittens, paotsin, squiddybot, peekaboo) to manually observe behavior before coding. Quick call to compare notes on each cat's pattern.	Oct 31	ALL
[Code] State/Reward Design — Design the reward structure and state-handling logic in <code>training.py</code> (how to interpret the 4-digit state int, distance-based shaping, catch bonus, step penalty, etc.). Document reasoning for later report use.	Nov 7	Member A
[Code] Q-Learning Core — Implement the main training loop: reset, explore/exploit (epsilon-greedy), Q-table update rule, decay schedule. Must run within the 5000-episode / 20-second constraint.	Nov 10	Member A
[Code] Hyperparameter Tuning & Trainer Cat Testing — Modify the Trainer cat	Nov 14	Member

Write a message...

Sonnet 5 Medium

Cloud is AI and can make mistakes. Please double-check responses.

Task division system

seperate column for name and dates are its July 13 today and this is due aug 7

Restructured timeline across twenty-five days with proportional deadline mapping.

Got it — here's the updated version with separate Date and Member columns, timeline adjusted for a July 13 → August 7 window:

Role	Description	Date	Member
[Setup] Environment Familiarization (shared)	Everyone runs <code>play.py</code> on all 5 known cats (batmeow, mittens, paotsin, squiddybot, peekaboo) to manually observe behavior before coding. Quick call to compare notes on each cat's pattern.	July 16	ALL
[Code] State/Reward Design	Design the reward structure and state-handling logic in <code>training.py</code> (interpreting the 4-digit state int, distance-based shaping, catch bonus, step penalty, etc.). Document reasoning for report use.	July 20	Member A
[Code] Q-Learning Core	Implement the main training loop: reset, explore/exploit	July	Member

Task division system

Share

can be 4 code roles and 4 writing roles

Architected balanced team structure pairing code and writing roles

Here's the restructured version with 4 code roles and 4 writing roles — one of each per member:

Role	Description	Date	Member
[Code] State/Reward Design	Design the reward structure and state-handling logic in <code>training.py</code> (Interpreting the 4-digit state int, distance-based shaping, catch bonus, step penalty, etc.). Document reasoning for later report use.	July 20	Member A
[Code] Q-Learning Core	Implement the main training loop: reset, explore/exploit (epsilon-greedy), Q-table update rule, decay schedule. Must run within the 5000-episode / 20-second constraint.	July 23	Member B
[Code] Hyperparameter Tuning & Trainer Cat Testing	Modify the Trainer cat in <code>cat_env.py</code> to build custom test scenarios (edge cases, adversarial movement). Tune learning rate, discount factor, epsilon decay. Run <code>bot.py</code> on all 5 known cats and log steps-to-catch.	July 27	Member C
[Code] Robustness / Generalization	Since 5 cats are hidden at env. <code>down</code> , stress-test generalization: vary starting positions, add noise via Trainer cat, adjust reward	July 29	Member D

ChatGPT

UpgradeShare

Pasted text(6).txt Document

can u simplify?

like bullet pooints and easy to understand

Yep! Here's a much simpler version with bullet points that are easier to memorize and explain.

1. Classifier Model

1.1 Problem

- The CatBot learns using Q-Learning.
- State = CatBot position + Cat position.
- Actions:
 - Up
 - Down
 - Left
 - Right

+ Ask anything

IV. VILLORENTE, KHYLE . - [AI TOOL(S) USED: GEMINI]

Describe briefly how you used the AI tool(s) in the completion of this assessment task. Examples include: brainstorming, idea generation.

1.1 AI used in Code Creation:

I used Gemini in helping me brainstorm how to make the test cats for edge/random cases.

1.2 AI used in Script Making:

None.

1.3 AI used in Overall Algorithm:

None.

In 2–3 sentences, reflect on how the use of AI contributed to your learning. If no AI was used, reflect on why you chose not to.

AI helped contribute to my learning of improving our bot through hyperparameter tuning. Since basing our bot on just the five provided cats do not account for other random behaviour, having the help of AI in building and suggesting other possible edge cases from ones I provided allowed me to test our bot and the adjustments I made to evaluate how well they worked or did not work.

Chat Logs:

okay can you help me edit the trainer cat code? i want to make a couple test cases for the behaviours:

1. what if the cat copies how the player moves
2. the cat moves in chess moves (like diagonally only, L-shape only etc)
3. what if the cat only stays on the edges
4. what if the cat only moves when the player is in a 2x2 area and other edge cases u think we should test.

This sounds like an incredibly fun way to stress-test your Q-learning agent! As an AI, I don't play video games, but I can certainly help you mathematically model and code these chaotic new behaviors.

Assuming your `cat_env.py` has a base `Cat` class (or similar structure) that dictates how cats move on the 8x8 grid (rows 0-7, cols 0-7), here is how you can implement these test cases.

1. The Mimic (CopyCat)

To copy the player's moves, the cat needs to know where the bot was last turn to calculate the direction it moved, and then apply that same direction to itself.

```
Python
class CopyCat:
    def __init__(self, start_row, start_col):
        self.r = start_row
        self.c = start_col
        self.last_bot_r = None
        self.last_bot_c = None

    def move(self, bot_r, bot_c):
        if self.last_bot_r is not None and self.last_bot_c is not None:
            # Calculate player's movement delta
            delta_r = bot_r - self.last_bot_r
            delta_c = bot_c - self.last_bot_c

            # Apply delta to cat, keeping it within 8x8 grid bounds
            self.r = max(0, min(7, self.r + delta_r))
            self.c = max(0, min(7, self.c + delta_c))

            # Update last seen bot position
            self.last_bot_r = bot_r
            self.last_bot_c = bot_c
```

2. The Grandmaster (ChessCat)

We can make a cat that randomly picks between moving like a Bishop (diagonals) or a Knight (L-shaped). This will completely mess with the bot's Manhattan-distance expectations.

```
Python
import random

class ChessCat:
    def __init__(self, start_row, start_col):
        self.r = start_row
        self.c = start_col

    def move(self, bot_r, bot_c):
        # Define Knight (L-shape) and Bishop (Diagonal 1-step) moves
        knight_moves = [(2,1), (2,-1), (-2,1), (-2,-1), (1,2), (1,-2), (-1,2), (-1,-2)]
        bishop_moves = [(1,1), (1,-1), (-1,1), (-1,-1)]

        all_moves = knight_moves + bishop_moves
        random.shuffle(all_moves) # Shuffle to pick a random valid move

        for dr, dc in all_moves:
            new_r = self.r + dr
            new_c = self.c + dc

            # Only take the move if it stays on the board
            if 0 <= new_r < 8 and 0 <= new_c < 8:
                self.r = new_r
                self.c = new_c
                break
```

3. The Wallhugger (EdgeCat)

This cat immediately bolts to the nearest edge and then strictly slides along the perimeter.

```
Python
class EdgeCat:
    def __init__(self, start_row, start_col):
        self.r = start_row
        self.c = start_col

    def move(self, bot_r, bot_c):
        # If not on an edge, move towards the nearest one
        if self.r not in (0, 7) and self.c not in (0, 7):
            dist_to_top, dist_to_bottom = self.r, 7 - self.r
            dist_to_left, dist_to_right = self.c, 7 - self.c

            min_dist = min(dist_to_top, dist_to_bottom, dist_to_left, dist_to_right)

            if min_dist == dist_to_top: self.r -= 1
            elif min_dist == dist_to_bottom: self.r += 1
            elif min_dist == dist_to_left: self.c -= 1
            else: self.c += 1
        else:
            # If already on an edge, slide randomly along it
            moves = []
            if self.c == 0 or self.c == 7:
                if self.c > 0: moves.append((0, -1))
                if self.c < 7: moves.append((0, 1))
            if self.c == 0 or self.c == 7:
                if self.r > 0: moves.append((-1, 0))
                if self.r < 7: moves.append((1, 0))

            if moves:
                dr, dc = random.choice(moves)
                self.r += dr
                self.c += dc
```

4. The Paranoiac (ProximityCat)

This cat literally just sleeps until CatBot enters a 2-tile radius (a 5 x 5 grid centered on the cat), at which point it runs directly away.

```
Python
class ProximityCat:
    def __init__(self, start_row, start_col):
        self.r = start_row
        self.c = start_col

    def move(self, bot_r, bot_c):
        # Check if bot is within 2 tiles (Chebyshev distance)
        if abs(self.r - bot_r) <= 2 and abs(self.c - bot_c) <= 2:
            # Run away horizontally
            if bot_c < self.c and self.c < 7: self.c += 1
            elif bot_c > self.c and self.c > 0: self.c -= 1

            # Run away vertically
            if bot_r < self.r and self.r < 7: self.r += 1
            elif bot_r > self.r and self.r > 0: self.r -= 1
```

5. My Suggested Edge Case: The Matador (SidestepCat)

A great edge case to test is a cat that waits until the bot is exactly 1 tile away, and then deliberately steps perpendicular to the bot's approach, like a bullfighter dodging a bull. It forces the bot to constantly change directions rather than just chasing in a straight line.

+ @ Ask Gemini Thinking

